

CURV AI — AWS Deployment Readiness Audit

Date: 2026-10-05 **Repository:** zohaibkhensa-oss/prachar (branch: main) **Target Region:** ap-south-1 (Mumbai) **AWS Budget:** \$25/month **Audit Type:** READ-ONLY (no resources created, no files modified)

1. Executive Summary

CURV AI is a production-grade AI advertising platform with a FastAPI backend, Next.js 15 frontend, Celery workers, PostgreSQL database, and Redis cache. The codebase is mature (621+ tests, 38 DB tables, 27 API routers, 17 organic channel adapters, 10 ads adapters) with a frozen architecture (ADR-0007).

Existing Terraform infrastructure code exists (17 .tf files) targeting ECS Fargate, RDS PostgreSQL 16, ElastiCache Redis, ALB, CloudFront, Route53, WAF, and CloudWatch. However, it contains **4 CRITICAL bugs** that will prevent `terraform apply` from succeeding.

The \$25/month budget is not achievable on the current AWS architecture — the minimum realistic AWS cost is ~\$130-160/month (single-AZ, smallest instances). The Terraform defaults would cost ~\$275-310/month.

Deployment readiness: READY WITH BLOCKERS — 4 critical Terraform fixes, 3 critical CI/CD fixes, and 2 high-priority security fixes are required before first deployment. No architectural redesign is needed.

2. Current Architecture

Component	Technology	Port	Notes
API	FastAPI, Python 3.12, uvicorn	8000	27 routers, ~177 endpoints, SSE streaming
Frontend	Next.js 15, React 19, TypeScript	3002	CSR SPA with API proxy, ~40 pages
Workers	Celery 5.4, Python 3.12	—	8 queue shards, 7-step weekly loop
Beat	celery-beat	—	3 scheduled jobs (dispatch, performance, anomalies)
Database	PostgreSQL 16	5432	38 tables, RLS, partitioned metrics
Cache	Redis 7	6379	Celery broker, idempotency, progress tracking
Storage	MinIO/S3	—	PDF reports, knowledge docs (metadata only currently)
AI/LLM	Groq, Anthropic, OpenAI, Gemini, fal.ai	—	External APIs, Wan 3.0 video gen
Payments	Stripe, Razorpay	—	Webhook verified
Email	Resend + SMTP fallback	—	External service

3. Current Deployment Architecture

Platform: Render (free tier staging) - `prachar-api` — Docker, `apps/api/Dockerfile`, health check `/health` - `prachar-web` — Docker, `apps/web-v2/Dockerfile` - **No worker or beat services defined** — weekly loop does NOT run on Render - Database: Supabase (external PostgreSQL) - Redis: Upstash (external) - Domain: `prachar-web.onrender.com` + `prachar-api.onrender.com`

CI/CD: GitHub Actions - `ci.yml` — tests (Python + web), runs on PR/push to main - `deploy.yml` — AWS deploy (BROKEN — wrong Dockerfile paths) - `security.yml` — audits (all `|| true` — failures silently ignored)

4. Repository Findings

4.1 Dockerfiles

- `apps/api/Dockerfile` — python:3.12-slim, installs shared+api+workers, port 8000, runs as **root** (MEDIUM)
- `apps/web-v2/Dockerfile` — node:20-slim, pnpm, `NEXT_PUBLIC_API_BASE` baked at build, port 3002, runs as **root** (MEDIUM)
- `apps/workers/Dockerfile` — python:3.12-slim + Chromium, runs as **root** (MEDIUM)
- **Worker Dockerfile has broken COPY paths** when built with repo-root context

4.2 Key Issues Found

1. **deploy.yml references nonexistent Dockerfiles** (`Dockerfile.api`, `Dockerfile.web`, `Dockerfile.worker`)
2. **render.yaml has no worker/beat services** — background jobs don't run on Render
3. **CI tests** `apps/web` (**v1**), not `apps/web-v2` which is the actual deploy target
4. **No** `pnpm-lock.yaml` in web-v2 — `--no-frozen-lockfile` used (supply-chain risk)
5. **Worker image doesn't install** `prachar-shared` **or** `prachar-api` but `loop.py` imports both

5. Environment Variable Inventory

Total: 46+ channel env vars + 20+ infrastructure env vars

Categorized Summary

Category	Count	AWS Storage
Database	3 (DATABASE_URL, DATABASE_URL_SYNC, REDIS_URL)	Secrets Manager
Authentication	4 (JWT_SECRET, JWT_REFRESH_SECRET, TOKEN_ENC_KEY, JWT_TTL)	Secrets Manager
OAuth (channels)	27 (13 providers x ~2 each)	Secrets Manager
AI/LLM	8 (Groq, Anthropic, OpenAI, Gemini, fal.ai, RunPod)	Secrets Manager
Payments	8 (Stripe, Razorpay keys + webhook secrets)	Secrets Manager
Storage	5 (S3 endpoint, bucket, keys, region)	Parameter Store + IAM role
Email	6 (Resend, SMTP, from address)	Secrets Manager / Parameter Store
Frontend	1 (NEXT_PUBLIC_API_BASE)	Build-time ARG
Monitoring	0 (NONE — gap)	—
Rate limiting	4	Parameter Store
Celery tuning	8	Parameter Store

Key Findings

- `.env` is properly gitignored 

- No secrets found in tracked files 
 - `.env.example` has duplicate Stripe/Razorpay blocks (LOW)
 - 8 env vars in Settings but missing from `.env.example` (documentation gap)
 - CORS origins are **hardcoded** in `main.py` — not env-driven (MEDIUM)
 - Domain appears as 5 different values across codebase: `prachar.ai`, `prachar.app`, `curv.ai`, `curv.app`, `onrender.com` (HIGH)
-

6. Database Findings

- **Technology:** PostgreSQL 16, SQLAlchemy 2 async (asyncpg)
- **Tables:** 38 (including partitioned `metric_events`)
- **Migrations:** 14 Alembic migrations
- **RLS:** Implemented on 27 tables, **missing on 11 tables** (MEDIUM)
- **Extensions:** None required (no pgvector needed yet)
- **Vector storage:** JSONB float arrays (not pgvector) — works but won't scale
- **Connection pooling:** `pool_size=25`, `max_overflow=50`, `pool_pre_ping`, `pool_recycle=1800`
- **PgBouncer-aware:** `statement_cache_size=0` when pooler detected

RLS Gaps (11 tables without policies)

- `council_sessions`, `director_opinions`, `consensus_decisions`, `campaign_scores`, `council_learnings`
- `knowledge_sources`, `knowledge_chunks`, `knowledge_embeddings`, `knowledge_attributions`
- `review_comments`, `review_versions`
- `campaign_performance` (no `tenant_id` column — needs schema change)

AWS Recommendation

- **RDS PostgreSQL 16** (db.t3.small for small prod)
 - **Migration complexity:** LOW-MEDIUM (1-2 days)
 - **Prerequisites:** Dedicated app DB role (non-owner for RLS), SSL verify-full, RLS remediation migration
-

7. Storage Findings

- **S3 usage:** Only PDF reports (`reports/{brand}/{week}.pdf`) — via inline boto3 in worker
- **Knowledge uploads:** Metadata stored in DB, **file bytes NOT persisted** (lost after processing)
- **Video generation:** Stored on `fal.ai` (external) or returned as base64 data URLs — NOT S3
- **Image generation:** Returned as base64 data URLs — NOT S3
- **No presigned URL endpoints** for downloading reports or assets
- **No file size limit** on `/knowledge/upload` — DoS risk
- **Terraform S3 config is good:** KMS encryption, versioning, lifecycle rules, public access block

S3 Architecture Recommendation

- Single bucket with prefix convention: `uploads/`, `knowledge/`, `reports/`, `creative/`, `video/`, `temp/`
- Add presigned URL endpoints for upload and download
- Move video/image storage from `fal.ai/base64` to S3

- Enforce 25MB file size limit on uploads
-

8. Realtime/Worker Findings

Celery Workers

- **8 queue shards** (loop-0 through loop-7) + dispatch, ingest, organic, ads, measure, creative, dlq
- **Beat schedule:** dispatch_due (60s), pull_daily_performance (03:00 UTC), check_anomalies (04:00 UTC)
- **7-step weekly loop:** measure → diagnose → regenerate → policy_check → publish → budget_realloc → report
- **Worker imports API code** (`loop.py` imports `prachar_api.runtime.realtime_feedback`) — worker image must contain API package

EventBus (TWO implementations, both in-memory)

1. **Runtime EventBus** — per-session `asyncio.Queue` , breaks with >1 API replica
2. **IntegrationEventBus** — per-process pub/sub for webhooks, no durability

SSE Endpoints (no WebSockets)

- `GET /runtime/stream` — AI session streaming
- `GET /audits/{id}/events` — audit progress
- Video generation blocks HTTP request up to 600s — **ALB will time out** (default 60s)

AWS Service Recommendations

- **ECS Fargate** — API + separate worker services per queue group
 - **EventBridge Scheduler** — replace celery-beat (more reliable singleton)
 - **Redis pub/sub** — back EventBus for multi-replica SSE
 - **SQS** — buffer webhooks for durability (optional)
 - **ALB idle timeout** — raise to ≥120s for SSE
-

9. Frontend Deployment Recommendation

Recommendation: ECS Fargate behind ALB (NOT S3+CloudFront)

The Next.js app is a CSR SPA that requires a running Node server for the `/api` rewrite proxy and 30-minute `proxyTimeout` for SSE video generation. Pure S3+CloudFront static hosting is **not viable** without moving the API proxy elsewhere.

- `output: "export"` is NOT set in next.config
- ~90 `"use client"` directives, zero `"use server"` or SSG functions
- `NEXT_PUBLIC_API_BASE` is baked at build time (Dockerfile ARG)
- No `next/image` optimization requirements (plain `<img>` tags)

Terraform gap: Current Terraform assumes S3+CloudFront for frontend. Needs an ECS Fargate service for web-v2 instead.

10. Backend Deployment Recommendation

Recommendation: ECS Fargate behind ALB

- **Lambda ruled out:** SSE streams up to 30 minutes exceed Lambda's 15-minute limit
- **App Runner ruled out:** No control over idle timeout for long SSE connections
- **EC2 unnecessary:** Stateless containerized API, Fargate is simpler
- **ECS Fargate fits exactly:** Port 8000, `/health` for ALB, supports long-lived connections

Caveats: - In-memory rate limiting won't scale >1 replica (use Redis or WAF) - In-memory EventBus breaks with >1 replica (use Redis pub/sub) - CORS is hardcoded (needs env-driven config)

11. Security Findings

Severity	Finding
CRITICAL	deploy.yml Dockerfile paths wrong — deploy will fail
HIGH	OAuth <code>state</code> = predictable <code>brand_id</code> , no nonce/session binding → CSRF/account-linking attack
HIGH	X/Twitter PKCE <code>code_verifier</code> never persisted — PKCE flow broken
HIGH	Domain drift: 5 different domains across codebase (prachar.ai, prachar.app, curv.ai, curv.app, onrender.com)
MEDIUM	Admin "ops" endpoints expose cross-tenant data to any tenant owner
MEDIUM	API tokens stored plaintext in-memory, not hashed, not wired to auth
MEDIUM	CORS origins hardcoded — new domains require code change
MEDIUM	Root user in all 3 Dockerfiles (api, worker, web-v2)
MEDIUM	In-memory rate limiting won't hold under multi-replica
MEDIUM	JWT via <code>?token=</code> query param for SSE (URL leakage)
MEDIUM	No production safety check for <code>change-me-*</code> placeholder secrets
MEDIUM	SNS alerts topic has no subscription — alarms fire to nobody
MEDIUM	No Sentry/error tracking — exceptions only in logs
MEDIUM	No OIDC in CI (long-lived AWS access keys)
MEDIUM	security.yml all audit steps <code> true</code> — failures silently ignored
LOW	Refresh tokens not rotated on use
LOW	<code>/metrics</code> and <code>/docs</code> publicly accessible
LOW	S3 public access block disabled on frontend bucket
LOW	<code>--no-frozen-lockfile</code> in web-v2 Dockerfile
LOW	<code>.env.example</code> duplicate Stripe/Razorpay blocks

Good security practices found: - AES-GCM encryption for OAuth tokens at rest ✅ - Security headers middleware (HSTS, CSP, nosniff, frame-deny) ✅ - Webhook signature verification (Stripe, Razorpay) ✅ - bcrypt password hashing ✅ - JWT typ claim enforcement ✅ - `.env` properly gitignored ✅ - No secrets in tracked files ✅

12. Observability Recommendation

Current State

- **Logging:** Plain text, `logging.basicConfig` INFO — NOT structured/JSON
- **Metrics:** `/metrics` Prometheus endpoint — in-memory only
- **Health:** `/health`, `/health/ready`, `/health/live` — solid
- **Error tracking:** NONE (no Sentry, no Rollbar, no Bugsnag)
- **Tracing:** NONE (no OpenTelemetry)

AWS Mapping

Application Need	AWS Service	Status
Log aggregation	CloudWatch Logs	Terraform configured (30d retention)
Metrics	CloudWatch Metrics	5 alarms configured
Error tracking	Sentry (external)	NOT CONFIGURED — gap
Health checks	ALB health check → <code>/health</code>	Configured
Dashboard	CloudWatch Dashboard	Configured
Alerts	SNS topic	Configured but NO subscription

Gaps to Fix

1. Add SNS email/PagerDuty subscription
2. Add Sentry DSN to env vars
3. Switch to structured JSON logging
4. Add worker queue depth + DLQ alarms
5. Add AI provider failure alarms

13. CI/CD Recommendation

Current Pipeline

- `ci.yml` — tests on PR/push (Python + web) 
- `deploy.yml` — AWS deploy (BROKEN)
- `security.yml` — audits (silently failing)

Recommended Strategy: GitHub Actions → OIDC → IAM Role

1. Create OIDC identity provider in AWS for GitHub
2. Create IAM role with Terraform permissions, trust policy for GitHub repo
3. Configure `aws-actions/configure-aws-credentials` with `role-to-assume`
4. **No long-lived AWS access keys in GitHub secrets**

Fixes Required

- Fix Dockerfile paths in deploy.yml
- Fix build contexts (web → `apps/web-v2`)
- Add test gate before deploy
- Add `pnpm-lock.yaml` and use `--frozen-lockfile`
- Remove `|| true` from security.yml
- Add Terraform remote state backend configuration

14. Domain/HTTPS Requirements

Current State (CHAOS)

Location	Domain
Terraform variables.tf	<code>prachar.ai</code>
CORS allowlist (main.py)	<code>prachar.app</code> , <code>prachar-web.onrender.com</code>
FastAPI metadata (main.py)	<code>curv.ai</code>
render.yaml	<code>curv.app</code>
<code>.env.example</code> EMAIL_FROM	<code>curv.app</code>
Dockerfile default	<code>prachar-api.onrender.com</code>

Recommendation

Pick ONE domain: `curv.app` - `app.curv.app` → CloudFront → frontend (ECS Fargate) - `api.curv.app` → ALB → API (ECS Fargate) - ACM certificate in us-east-1 (CloudFront) + **separate regional cert in ap-south-1** (ALB) - Route53 hosted zone for `curv.app` - Update CORS, WEB_URL, EMAIL_FROM, OAuth redirect URIs in all provider consoles

OAuth Redirect URIs (must be updated in each provider console)

`https://app.curv.app/app/connections/{channel}/callback`

For: Google, YouTube, Meta (FB/IG/WhatsApp), TikTok, LinkedIn, X, Pinterest, LINE, VK, Reddit, Naver

15. AWS Target Architecture

```
CURV AI
|
+-- Frontend (Next.js 15) → ECS Fargate → ALB → CloudFront
|
+-- CDN / HTTPS → CloudFront + ACM + Route53
|
+-- API (FastAPI) → ECS Fargate → ALB (port 8000)
|
+-- Workers (Celery) → ECS Fargate (separate services per queue)
|
+-- Beat (Scheduler) → EventBridge Scheduler (singleton)
|
+-- Database → RDS PostgreSQL 16 (Multi-AZ)
|
+-- Cache → ElastiCache Redis 7.1 (replication group)
|
+-- Object Storage → S3 (KMS-encrypted, lifecycle rules)
|
+-- Queue/Event System → Redis (broker) + SQS (optional webhook buffer)
|
+-- AI Providers → External (Groq, Anthropic, OpenAI, Gemini, fal.ai)
|
+-- External Integrations → OAuth webhooks via ALB
|
+-- Monitoring → CloudWatch Logs + Alarms + Dashboard + SNS
|
+-- CI/CD → GitHub Actions → OIDC → IAM Role → ECS + ECR
```

16. AWS Service-by-Service Mapping

AWS Service	Why Needed	Used By	Mandatory?	Est. Cost/mo	Complexity
ECS Fargate	Container hosting	API, Worker, Beat, Web	Yes	\$35-105	Low
RDS PostgreSQL 16	Database	API, Workers	Yes	\$45-90	Medium
ElastiCache Redis	Cache + broker	API, Workers	Yes	\$25-50	Low
ALB	Load balancing	API, Web	Yes	\$25	Low
NAT Gateway	Private subnet egress	All	Yes	\$35	Low
S3	Object storage	Workers (reports)	Yes	\$2	Low
CloudFront	CDN + HTTPS	Frontend	Yes	\$5	Low
Route53	DNS	All	Yes	\$1	Low
ACM	TLS certificates	ALB, CloudFront	Yes	\$0	Low
ECR	Container registry	CI/CD	Yes	\$1	Low
Secrets Manager	Secret storage	ECS tasks	Yes	\$3	Low
KMS	Encryption keys	S3, RDS, Secrets	Yes	\$1	Low
CloudWatch	Logs + alarms	All	Yes	\$5	Low
SNS	Alert notifications	CloudWatch	Yes	\$0	Low
WAF	Web firewall	ALB	Optional	\$10	Low
IAM	Access control	All	Yes	\$0	Medium

17. Cost Estimate

\$25 Budget Reality Check

The \$25/month budget is **NOT** achievable on AWS with the current architecture.

Tier	Monthly Cost	Feasibility
\$25 budget	❌ Impossible	Would need single EC2 VM with Docker or free-tier external services
Dev/Staging (minimal)	~\$130-160	Single-AZ, t3.micro, 1 Fargate task, no WAF
Small Production	~\$275-310	Multi-AZ, t3.small, 1 API + 1 worker, WAF, CloudFront
Growth Production	~\$1,220-1,280	Multi-AZ, r6g.large, 2 API + 2 worker, full stack

Cost Breakdown (Small Production)

Item	Est./mo
Fargate API (1 vCPU/2GB x1)	\$35
Fargate Worker (0.5 vCPU/1GB x1)	\$18
RDS db.t3.small Multi-AZ + 20GB	\$85
ElastiCache cache.t3.small x2	\$50
ALB + LCUs	\$25
NAT Gateway + processing	\$35
S3 + CloudFront + Route53	\$8
Secrets Manager + KMS	\$4
WAF	\$10
CloudWatch + SNS	\$5
Total	~\$275/mo

What Can Run Within \$25

- Single t3.small EC2 instance with Docker (\$15/mo)
- All services (API, worker, beat, web, Postgres, Redis) in containers on one VM
- No Multi-AZ, no ALB, no WAF, no CloudFront
- **NOT recommended for production** — single point of failure

Recommendation

Start with **dev/staging at ~\$130/mo** (single-AZ, minimal instances) to validate the deployment, then scale to small production (~\$275/mo) when ready for users.

18. Migration Plan

STAGE 0: Audit Complete

STAGE 1: AWS Foundation (USER ACTION)

- Create AWS account, configure CLI, set up billing alerts

STAGE 2: Fix Terraform (CODE CHANGES)

- Fix 4 critical Terraform bugs (regional cert, Fargate memory, DB password, state backend)

STAGE 3: Fix CI/CD (CODE CHANGES)

- Fix Dockerfile paths, add OIDC, add test gate

STAGE 4: Fix Application Issues (CODE CHANGES)

- Fix OAuth state, PKCE, domain unification, CORS, Docker security

STAGE 5: Database Remediation (MIGRATION)

- Add RLS policies to 11 tables, add tenant_id to campaign_performance

STAGE 6: Infrastructure Deployment

- Bootstrap state, terraform init/plan/apply, populate secrets, run migrations

STAGE 7: Application Deployment

- Build/push images, deploy ECS services, verify health

STAGE 8: DNS Cutover

- Update Route53, OAuth redirect URIs, CORS origins

STAGE 9: Monitoring & Rollback

- SNS subscription, error tracking, rollback runbook

STAGE 10: E2E Testing

- Promote audit scripts to Playwright tests in CI

19. Rollback Strategy

- **ECS rollback:** Revert to previous ECR image tag in task definition
- **Database rollback:** `alembic downgrade -1` (test on staging first)
- **DNS rollback:** Revert Route53 records to Render (keep Render running for 7 days post-cutover)

- **Terraform rollback:** `terraform destroy` for staging, keep production state
- **Keep Render running** for 7 days as fallback during DNS cutover

20. Playwright/E2E Readiness

Current State

- **No** `playwright.config.ts` **exists**
- `apps/web-v2/e2e-audit.mjs` — 500-line ad-hoc chromium script covering real workflows
- `apps/web-v2/qa-audit.mjs` — 34-route render/error audit
- `apps/web/screenshot.config.ts` — 24 screenshot "tests" with no real assertions
- **No E2E in CI pipeline**

Coverage Assessment

Workflow	Covered?
Authentication (login, register, social)	✓
Onboarding	✓
Dashboard + Orb chat	✓
Campaign creation	✓
Creative studio	✓
Performance	✓
Knowledge hub	✓
Integrations/connections	✓
Settings (profile, billing, API, notifications)	✓
Timeline/review queue	✓
Labs pages (11)	✓
OAuth callback flow	✗
File uploads	✗
Responsive mobile UI	Partial (desktop + mobile screenshots)

Recommendation

Convert `e2e-audit.mjs` to proper Playwright test specs, add to CI, add missing OAuth and file upload tests.

21. Architecture Freeze Compliance

COMPLIANT ✓

- Architecture freeze declared 2026-08-02 (ADR-0007)
- `test_architecture_freeze.py` enforces: single Runtime/Planner/Composer, no shared→api imports, approved package list

- AWS deployment introduces **no new core abstractions**
- ECS/RDS/ElastiCache are infrastructure, not application architecture
- 5 pre-existing shared→api violations are documented grandfathered debt
- Two soft flags:
- Freeze says "Secrets only via env/SSM" — Secrets Manager is functionally equivalent (RECOMMENDATION/INFERENCE: update wording)
- Terraform IAM/secrets don't touch app abstractions — safe

22. Risks

Risk	Severity	Mitigation
\$25 budget insufficient for AWS	HIGH	Start with single-VM or external free tiers
SSE breaks with >1 API replica	HIGH	Use Redis pub/sub or sticky sessions
Video gen blocks HTTP 600s	HIGH	Move to Celery task + polling
11 tables missing RLS	MEDIUM	Add remediation migration before cutover
No error tracking in production	MEDIUM	Add Sentry before launch
Domain drift causes CORS/OAuth failures	HIGH	Unify to curv.app before cutover
Worker image missing dependencies	HIGH	Fix Dockerfile to install shared+api
No Celery beat in Terraform	HIGH	Add EventBridge Scheduler
In-memory rate limiting	MEDIUM	Use Redis-backed or WAF
Terraform state backend not bootstrapped	CRITICAL	Create S3+DynamoDB out-of-band first

23. Blockers

1. **CRITICAL:** Terraform ALB HTTPS listener uses us-east-1 cert (needs regional cert)
2. **CRITICAL:** Terraform `ecs_api_memory = 1048` is invalid Fargate combo
3. **CRITICAL:** Terraform DB password may contain URL-breaking characters
4. **CRITICAL:** Terraform state backend not bootstrapped (chicken-and-egg)
5. **CRITICAL:** `deploy.yml` Dockerfile paths don't exist
6. **HIGH:** No Celery beat service in Terraform
7. **HIGH:** Worker Dockerfile doesn't install required packages
8. **HIGH:** OAuth state parameter is predictable (security)
9. **HIGH:** Domain not unified (5 different domains in use)
10. **HIGH:** AWS credentials not configured (USER ACTION)

24. Prerequisites

- AWS account with administrative access (USER ACTION)
- AWS CLI configured locally (USER ACTION)
- Domain `curv.app` registered and transferable to Route53 (USER ACTION)

- Terraform v1.16+ installed ✓
 - Docker installed ✓
 - All OAuth provider consoles updated with new redirect URIs (USER ACTION)
 - Render kept running as fallback for 7 days post-cutover
-

25. Exact Next Deployment Steps

1. **USER ACTION:** Configure AWS CLI (`aws configure` or `aws login`)
 2. **USER ACTION:** Register/transfer `curv.app` domain to Route53
 3. Fix 4 critical Terraform bugs (regional cert, Fargate memory, DB password, state bootstrap)
 4. Fix `deploy.yml` Dockerfile paths and build contexts
 5. Fix worker Dockerfile to install `prachar-shared` + `prachar-api`
 6. Add Celery beat service to Terraform (EventBridge Scheduler)
 7. Unify domain to `curv.app` across all code (CORS, `WEB_URL`, `EMAIL_FROM`, Terraform vars)
 8. Fix OAuth state parameter (signed JWT with nonce)
 9. Fix X/Twitter PKCE (persist `code_verifier`)
 10. Add non-root USER to all Dockerfiles
 11. Bootstrap S3 state bucket + DynamoDB lock table manually
 12. Create `terraform.tfvars` with `curv.app` domain, `t3.small` instances
 13. Run `terraform init` + `terraform plan` + `terraform apply`
 14. Populate Secrets Manager with real values (out-of-band)
 15. Build + push Docker images to ECR
 16. Run Alembic migrations via one-off ECS task
 17. Deploy ECS services (API, worker, beat, web)
 18. Verify health endpoints
 19. Update OAuth redirect URIs in all provider consoles
 20. DNS cutover to Route53
-

DEPLOYMENT READINESS

READY WITH BLOCKERS

4 critical Terraform fixes + 3 critical CI/CD fixes + 2 high-priority security fixes required before first AWS deployment. No architectural redesign needed.

TOP 10 ACTIONS REQUIRED BEFORE FIRST AWS DEPLOYMENT

1. **Fix Terraform ALB cert** — add regional ACM cert in `ap-south-1` (separate from `us-east-1` CloudFront cert)
2. **Fix Terraform Fargate memory** — change `ecs_api_memory` default from 1048 to 2048
3. **Fix Terraform DB password** — use alphanumeric-only random password
4. **Bootstrap Terraform state** — create S3 bucket + DynamoDB table out-of-band before first `terraform init`
5. **Fix `deploy.yml`** — correct Dockerfile paths to `apps/api/Dockerfile`, `apps/web-v2/Dockerfile`, `apps/workers/Dockerfile`
6. **Fix worker Dockerfile** — install `prachar-shared` and `prachar-api` packages
7. **Add Celery beat** — EventBridge Scheduler or singleton Fargate task (weekly loop won't run without it)

8. **Unify domain** — pick `curv.app`, update CORS, WEB_URL, EMAIL_FROM, Terraform vars, OAuth redirect URIs
 9. **Fix OAuth state** — replace bare `brand_id` with signed JWT containing nonce + `brand_id` + `user_id`
 10. **Add frontend ECS service** — Terraform assumes S3+CloudFront but Next.js needs a running Node server
-

AWS Permissions Required for Next Phase (DO NOT CREATE YET)

```
# Terraform deployment role
iam:CreateRole, iam:PutRolePolicy, iam:AttachRolePolicy, iam:CreatePolicy
ec2:CreateVpc, ec2:CreateSubnet, ec2:CreateSecurityGroup, ec2:CreateNatGateway
ecs:CreateCluster, ecs:RegisterTaskDefinition, ecs:CreateService
rds:CreateDBInstance, rds:CreateDBSubnetGroup
elasticache:CreateReplicationGroup, elasticache:CreateCacheSubnetGroup
s3:CreateBucket, s3:PutBucketPolicy, s3:PutEncryptionConfiguration
cloudfront:CreateDistribution, cloudfront:UpdateDistribution
route53:CreateHostedZone, route53:ChangeResourceRecordSets
acm:RequestCertificate, acm:DescribeCertificate
elasticloadbalancing:CreateLoadBalancer, elasticloadbalancing:CreateListener
secretsmanager:CreateSecret, secretsmanager:PutSecretValue
ecr:CreateRepository, ecr:PutLifecyclePolicy
logs:CreateLogGroup, logs:PutRetentionPolicy
sns:CreateTopic, sns:Subscribe
wafv2:CreateWebACL, wafv2:AssociateWebACL
kms:CreateKey, kms:CreateAlias
```

Recommended: Create a `terraform-deployer` IAM role, assume via OIDC from GitHub Actions. Do NOT create long-lived access keys.